

TRABAJO DE FIN DE GRADO

Ecosistema de Domótica Local Impulsado por Agentes de IA

Luxe Core AI

Autor: Jesús Fernández López

Titulación: Grado en Ing. Informática

Tutor: Dr. Rafael Muñoz Salinas

Universidad de Córdoba — EPSC

Córdoba, junio de 2026



1. Introducción

Resumen Ejecutivo

2. Problema y Objetivos

Problema y Objetivo General

Objetivos Específicos

3. Restricciones y Metodología

Restricciones y Arq. Local

Metodología

4. Especificación del Sistema

Arquitectura de 3 Capas

Model Router v2

Comparativa de Modelos

Asesores de Confort

Auto-Recuperación

5. Integración de Hardware

Tuya: Control Local

Sensores BLE Xiaomi

FanLamp: Ing. Inversa

6. Validación y Pruebas

Pruebas y Resultados

7. Demostración

Vídeo de Demostración

Telegram + Dashboard

8. Conclusiones

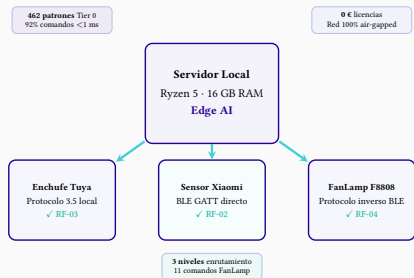
Logros Alcanzados

Trabajo Futuro

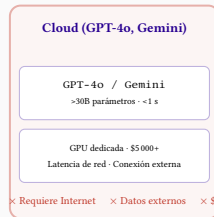
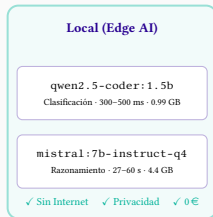
Privacidad por Diseño

Cierre

► Resumen Ejecutivo y Motivación



Modelos pequeños en CPU vs grandes en GPU



¿Qué es Luxe Core AI?

Sistema 100 % local de control de confort térmico orquestado por IA en el *edge*. **462 patrones** Tier 0 (92 %, <1 ms) · **0 €** licencias · Red air-gapped.

Principio rector

La tecnología del hogar no debería exigir ceder la privacidad a cambio de comodidad.

► Problema y Objetivo General

El problema

La domótica actual depende de servicios cloud: sin Internet, el hogar inteligente se vuelve inerte. Los datos de comportamiento viajan a servidores de terceros.

La respuesta

Sistema 100 % local de control de confort térmico orquestado por IA en el *edge*. Interoperabilidad real sin modificar hardware.

Objetivo general

Diseñar, implementar y validar un sistema de monitorización y control de confort térmico que opere de forma autónoma en una red local privada, eliminando cualquier dependencia de servicios en la nube.

Coste cero

0 € en licencias, APIs externas o suscripciones

Reproducibilidad

Mismo hardware → replicar con la documentación

Hardware real

Pruebas sobre dispositivos físicos, no simulación

► Objetivos Específicos e Indicadores

1. Arquitectura de microservicios local✓

OpenClaw systemd · Interfaces desacopladas · Diagramas de despliegue

2. Control HVAC por bus LIN~ Pendiente

ESP32 + LINTTL3 · Firmware C++20 · Comunicación bidireccional

3. Integración IoT heterogénea✓ 3/3

Xiaomi BLE (RF-02) · Tuya local (RF-03) · FanLamp (RF-04)

Cobertura

92%

comandos

Latencia

<2 s

red local

Disponibilidad

99%

24 h sin caídas

Aislamiento

100%

air-gapped

► Restricciones y Arquitectura Local

Restricciones de partida

- Un curso académico → MVP funcional
- Presupuesto: **56,50 €** — hardware de consumo
- Ingeniería inversa como alternativa a la inversión

Infraestructura

- Fase 1: servidores UCO con GPU (prototipado)
- Fase 2: torre Ryzen 5, 16 GB RAM, 100 % local

Arquitectura de red air-gapped

- Router TL-MR6400 sin tarjeta SIM — sin salida WAN
- Segmento 192.168.1.0/24 aislado, IPs estáticas
- Tráfico no puede abandonar la subred
- Verificable: retirar la SIM demuestra el diseño

Principio clave

Domótica inteligente $\neq$ grandes inversiones.
La interoperabilidad no depende de la nube:
depende del software.

► Metodología: Docs-as-Code y Git

Docs as Code

- Memoria $\LaTeX$ + código en mismo repositorio
- 9 ADR (*Architecture Decision Records*)
- Diagramas TikZ versionados
- Trazabilidad: diseño $\leftrightarrow$ código

Uso de IA en el desarrollo

- **OpenCode**: asistente de codificación (software libre)
- **DeepSeek V4**: modelo de inferencia vía API
- Toda línea revisada y validada por el autor

Gestión

SSH con llaves RSA, Cockpit, systemd.

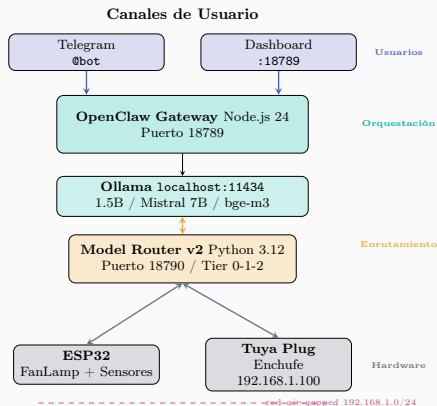
► Arquitectura de 3 Capas

Servidor de orquestación

- Torre Ryzen 5, 16 GB RAM, Ubuntu Server headless
- OpenClaw Gateway (Node.js 24, puerto 18789)
- Ollama: qwen2.5 1.5B + mistral 7B + bge-m3
- Drivers Python (Tuya, BLE, FanLamp)

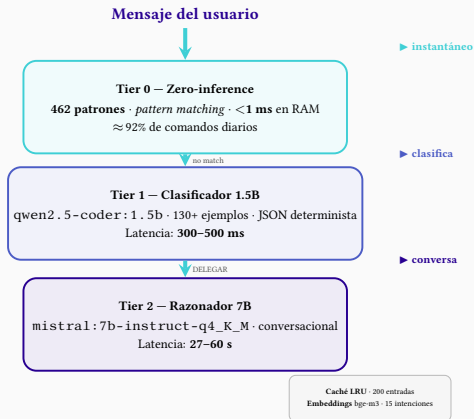
Nodo firmware

- ESP32 — puente BLE FanLamp
- Futuro: LIN HVAC con el mismo hardware



Comunicación exclusiva sobre 192.168.1.0/24. Ningún dato abandona el perímetro local.

► Model Router v2: Enrutamiento en 3 Niveles



Ejemplos

Com.	Niv.	Lat.
«vel.3»	T0	1,5 s
«casa»	T0	<1 ms
«enciendeT1	T1	300 ms
«buenos días»	T2	27–60 s

Circuit Breaker

Si el 1.5B falla, deriva al Tier 2.

► Selección de Modelos: Pruebas y Decisión

Proceso de evaluación

Tres fases de pruebas sobre Ryzen 5 (16 GB RAM, sin GPU)

- Corpus de 500 comandos reales
- Modelos probados: Qwen2.5 1.5B, Mistral 7B, Qwen3.5 4B
- Criterios: latencia, fiabilidad, consumo de recursos

Qwen3.5: no viable en CPU

Versión reciente con modo *thinking* forzado (no opcional). Resultado: 3–12 min por consulta en CPU. Diseñado para GPU.

Arquitectura seleccionada

Tier 1 – Qwen2.5 1.5B (determinista)

- 0,3–0,5 s por consulta
- Clasificación de comandos

Tier 2 – Mistral 7B Q4 (conversacional)

- 27–60 s por consulta
- Razonamiento complejo

Ventaja clave

Model-agnostic: la arquitectura permite cambiar modelos sin modificaciones. Si un modelo futuro funciona mejor en CPU, se integra directamente.

► Asesores de Confort Térmico

Evaluación del confort

El sistema evalúa continuamente:

- Condiciones interiores (temperatura, humedad)
- Condiciones exteriores (clima, radiación solar)
- Presencia de usuarios

Toma de decisiones

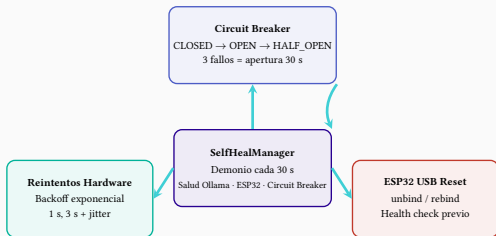
Basado en umbrales definidos:

- Sugiere acciones al usuario
- Ejecuta automáticamente si el usuario acepta
- Respeta siempre la decisión del usuario

Principio clave

El sistema sugiere, el usuario decide.

► Auto-Recuperación y Tolerancia a Fallos



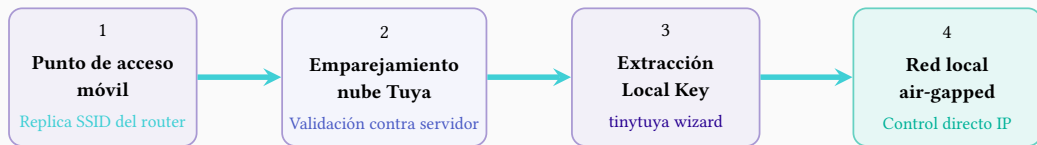
Flujo de recuperación

SelfHealManager verifica cada 30 s la salud de Ollama, Circuit Breaker y ESP32. Ante un fallo:

1. **Circuit Breaker** aísla el servicio (CLOSED → OPEN)
2. **Reintentos** con backoff + jitter (1 s, 3 s)
3. Si persiste: **reset USB** del ESP32

Sin intervención humana.

► Integración Tuya: Control Local y Hotspot Spoofing



Local Key nunca abandona el servidor – Control 100% local sin depender de Tuya



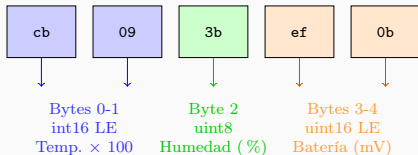
Protocolo tinytuya 3.5

Local Key en servidor · Respuesta <2 s

Control ON/OFF validado en red air-gapped

► Sensores BLE Xiaomi: Lectura GATT Directa

Payload GATT (5 bytes): cb 09 3b ef 0b



0x09cb = 2507

0x3b = 59

0x0bef = 3055

25.07°C

59 % HR

3055 mV

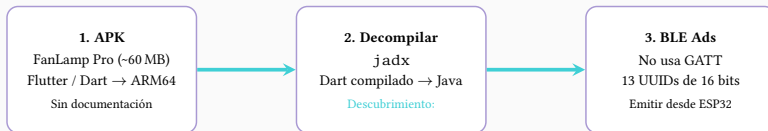


Xiaomi LYWSD03MMC

Lectura GATT directa

Decodificación de temperatura, humedad y batería
directamente del sensor, sin *bind key* ·
Sin nube Xiaomi

► FanLamp F8808: Ingeniería Inversa del Protocolo



11 comandos verificados

OFF
Apagado total

FAN
ON / OFF

LIGHT
ON / OFF

1-5
Velocidades

NIGHT
Modo noche



Descubrimiento clave

APK FanLamp Pro decompilado (jadx)

No usa GATT, sino anuncios BLE

11 comandos mapeados — emitidos desde ESP32 vía USB

► Validación: Pruebas y Resultados

Pruebas sobre hardware real:

1. **Tuya:** conectividad IP, ON/OFF → ✓
2. **BLE Xiaomi:** GATT, decodificación → ✓
3. **FanLamp:** 11 comandos físicos → ✓
4. **Model Router:** 3 niveles → ✓
5. **Circuit Breaker:** fallo + recuperación → ✓

Suite automatizada

tests/test_integration.py

5 subsistemas. No destructiva: solo comandos de lectura.

Cobertura Tier 0 (462 patrones):

- Iluminación (35 variantes): 100 %
- Ventilador (50 variantes): 100 %
- Escenas (40 variantes): 100 %
- Consultas estado (15 variantes): 100 %

Disponibilidad

model-router · sensor-daemon ·
openclaw-gateway

Operativos 24 h sin caídas.

► Demostración: Test de Integración Completa

Panel multimedia



Protocolo de Contingencia

Respaldo 1 – Local

► Abrir archivo local

video_demo.mp4

Respaldo 2 – YouTube

► Ver en YouTube

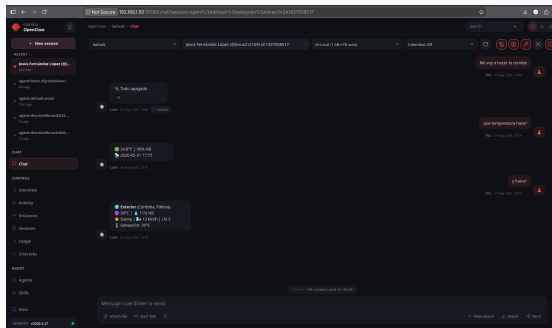
youtu.be/___rJUl pLXP4



► Demostración Práctica: Telegram + Dashboard



Conversación real vía Telegram



Dashboard web de OpenClaw

Comandos en lenguaje natural, interpretados en local,
ejecutados en hardware real. Ningún mensaje abandona la subred.

► Logros Alcanzados vs Rúbrica

Verificados sobre hardware real

- **Arquitectura 3 capas escalable**
- **Control local Tuya** — ON/OFF <2 s
- **Lectura BLE Xiaomi** — GATT sin nube
- **FanLamp F8808** — 11 comandos
- **Model Router v2** — 462 patrones, 92 % cobertura
- **Demos funcionando** 24 h, sin caídas

Estado de los módulos

Módulo	Estado	Obs.
Tuya Plug	✓	Air-gapped
BLE Xiaomi	✓	GATT directo
FanLamp	✓	11 comandos
OpenClaw	✓	Puerto 18789
Model Router	✓	3 niveles
ESP32 HVAC	Pendiente	Firmware C++20

5/6 módulos. 0 € en licencias.

▶ Trabajo Futuro

Corto plazo (semanas)

- ▶ Firmware HVAC ESP32 + LIN (prioridad inmediata)
- ▶ Bucle de control con OpenClaw sensores
- ▶ *Streaming* SSE para respuestas Tier 2
- ▶ Containerización Docker

Medio plazo (meses)

- ▶ Expansión dashboard: analíticas energéticas, ASHRAE 55
- ▶ Base de datos vectorial (Qdrant/ChromaDB)
- ▶ Memoria conversacional con bge-m3

Largo plazo (investigación)

- ▶ Aprendizaje adaptativo (*reinforcement learning*)
- ▶ Nuevos protocolos: Zigbee, Z-Wave
- ▶ Re-evaluación de modelos (Qwen3.5, Llama 4, DeepSeek)
- ▶ *Fine-tuning* con datos reales de uso doméstico

► Privacidad por Diseño: Reflexión

El principio fundamental

La tecnología del hogar no debería exigir ceder la privacidad a cambio de comodidad.

Lo que hemos comprobado:

- Es posible tener **ambas cosas** con hardware de consumo
- La arquitectura *air-gapped* es viable y verificable
- Los modelos pequeños en local **funcionan**
- La interoperabilidad no depende de la nube: depende del software

Más allá de lo técnico

La decisión de mantener todo en local fue una cuestión de principios.

Y en pocos años...

Modelos ligeros con capacidad cercana a los grandes cloud:

«¿se puede en local?» → «¿por qué en la nube?»



Luxe Core AI

Muchas gracias por su atención.

Quedo a disposición del ilustre tribunal
para resolver las cuestiones que consideren oportunas.

Jesús Fernández López
Grado en Ing. Informática — UCO
Junio 2026

